

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

**Nº 50391 - 2026 *Testing System for Java Payara APIs:
An Automated Regression and CI/CD Framework for
the GNSS EPOS API***

Author:

Eduardo Pires Valentim de Almeida Abrantes

Supervisor and Co-Supervisor:

**Professor Paul Andrew Crocker
Vasco Senra**

June 19, 2026

Acknowledgments

The completion of this project, and indeed of a large part of my academic journey, would not have been possible without the help, guidance and patience of many people. I would first like to thank my supervisor, Professor Paul Andrew Crocker, and my co-supervisor, Vasco Senra, for proposing this project and for the technical direction, the critical reviews and the trust placed in me throughout the development of the work described in this report.

A special word of gratitude goes to the staff of the SEGAL Laboratory at the Universidade da Beira Interior, Luís Carvalho and Fernando Geraldes, for the patient support with the virtual machines, the internal network and the VPN access that made the laboratory's infrastructure reachable from outside the lab and that, in practice, made the work described in this report possible.

I would also like to thank the colleagues and researchers I had the opportunity to work with at the laboratory, for the many discussions about the legacy code base, the refactoring strategy and the painful reality of regression testing when no two responses are quite alike. Those conversations shaped many of the choices presented in the following chapters.

Finally, to my family, for the constant and unconditional support that allowed me to reach this point, and to the friends and colleagues who were patient enough to listen to long monologues about differential testing, JAX-RS endpoints and YAML pipelines: thank you.

Contents

Contents	iii
List of Figures	v
1 Introduction	1
1.1 Framing	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Organization of the Document	3
1.5 Repository Access	3
2 State of the Art	5
2.1 Introduction	5
2.2 Test Frameworks for the Java Ecosystem	5
2.2.1 Differential Testing	6
2.3 Continuous Integration and Continuous Delivery	6
2.3.1 Foundations	6
2.3.2 Platforms	7
2.4 Conclusion	8
3 Technologies and Tools Used	9
3.1 Introduction	9
3.2 APIs, Serialization Formats and Specification	9
3.2.1 REST and the Multi-format Response	9
3.2.2 JavaScript Object Notation (JSON) and eXtensible Markup Language (XML)	10
3.2.3 Comma-Separated Values (CSV)	10
3.2.4 OpenAPI Specification (OpenAPI) and Swagger User Interface (UI)	10
3.3 Java and Jakarta EE	11
3.4 Structured Query Language (SQL) and the Database Layer	12
3.4.1 Jakarta Persistence API (JPA), Hibernate and the Persistence Unit	12

3.4.2	The European Plate Observing System (EPOS) Database in Practice	13
3.5	Servers, Tooling and Network	13
3.6	Conclusion	14
4	Methodology and Architecture	17
4.1	The Legacy Application Programming Interface (API) and the Need for a Safety Net	17
4.2	Iterative Methodology	17
4.3	Prototype Applications	18
4.4	Architecture of the Differential Testing Framework	18
5	Use Case: EPOS API	21
5.1	Architecture of api_v4.0	21
5.1.1	Endpoints and Output Formats	22
5.1.2	Persistence	22
5.2	The Continuous Integration (CI)/Continuous Delivery (CD) Pipeline	24
5.3	The Code-Level Diff: DiffHarness.java	26
5.4	The Hypertext Transfer Protocol (HTTP)-Level Diff: HttpDiff.java	27
5.5	Reports and Findings	28
6	Conclusions and Future Work	29
6.1	Main Conclusions	29
6.2	Future Work	30
	Bibliography	31

List of Figures

3.1	Multi-format content negotiation in <code>api_v4.0</code> . The same Java resource method is reused across four serializations, selected by a path parameter, and the differential harness must compare the body of each format separately.	11
4.1	The <i>Contexts and Dependency Injection</i> (CDI) + Jakarta RESTful Web Services (JAX-RS) exception-handling flow used by <code>syncspace-jakarta-app</code> and partly inherited by <code>api_v4.0</code> . Any exception thrown inside a resource or service bean is intercepted by the most specific registered <code>ExceptionHandler</code> , which produces a stable JSON error body.	19
4.2	Overall architecture of the differential testing framework. Solid arrows denote control flow inside the pipeline; dashed arrows denote artifact or data access. Both diff jobs run a three-way comparison between the HEAD build and two baselines, the immediate parent commit and a fixed reference commit, all pointed at the same database fixture.	20
5.1	Layered architecture of <code>api_v4.0</code> . Solid arrows denote ordinary call/data flow; dashed arrows denote configuration and infrastructure binding established at deployment time.	22
5.2	Simplified view of the core EPOS data model, with representative columns as they appear in the live PostgreSQL schema. Green entities carry domain data; purple entities are controlled-vocabulary lookups (note that <code>country.iso_code</code> is the natural primary key, not a surrogate <code>id</code>); grey entities are junction tables that materialise the many-to-many relationships of the model. Arrows point from the “one” side to the “many” side of the relationship.	23

- 5.3 Phases of `HttpDiff.java`. The harness boots one Payara Micro per available Web Application Archive (WAR), the HEAD build and whichever of the recent/fixed baselines were provided, waits for all of them to be ready, fires every test case against every instance in parallel, then performs a three-way comparison and emits one diff report per baseline. All instances share the same real PostgreSQL fixture over the laboratory Virtual Private Network (VPN). 27

List of Code Listings

5.1 Condensed view of the production <code>.gitlab-ci.yml</code> , three stages and four jobs.	24
--	----

Acronyms

API	Application Programming Interface
CDI	<i>Contexts and Dependency Injection</i>
CI	Continuous Integration
CD	Continuous Delivery
CSV	Comma-Separated Values
DSL	Domain Specific Language
EPOS	European Plate Observing System
GNSS	Global Navigation Satellite System
HTTP	Hypertext Transfer Protocol
ID	Identifier
JAR	Java Archive
JAX-RS	Jakarta RESTful Web Services
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object-Relational Mapping
OpenAPI	OpenAPI Specification
POJO	Plain Old Java Object
REST	Representational State Transfer
RINEX	Receiver Independent Exchange Format
SEGAL	Space and Earth Geodetic Analysis Laboratory
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UI	User Interface
URL	Uniform Resource Locator
VM	Virtual Machine

VPN	Virtual Private Network
WAR	Web Application Archive
XML	eXtensible Markup Language
YAML	<i>YAML Ain't Markup Language</i>

Chapter

1

Introduction

1.1 Framing

This document describes the work developed in the context of the final-year project of the Computer Science and Engineering programme of the Universidade da Beira Interior (UBI), under the project code PC26-4 of the 2025/2026 academic year. The project was proposed by, and carried out at, the Space and Earth Geodetic Analysis Laboratory (SEGAL) laboratory of UBI, under the supervision of Professor Paul Andrew Crocker and the co-supervision of Vasco Senra, with day-to-day infrastructure support from Luís Carvalho and Fernando Geraldês (responsible for the laboratory's virtual machines, internal network and Virtual Private Network (VPN) access).

SEGAL is the research laboratory of UBI that operates and maintains a Global Navigation Satellite System (GNSS) data and metadata service contributing to the European Plate Observing System (EPOS), a pan-European research infrastructure focused on solid Earth science [1, 2]. As part of the EPOS Thematic Core Service for GNSS data and products, SEGAL runs a Jakarta EE Representational State Transfer (REST) Application Programming Interface (API) that exposes GNSS station metadata and observation files (in the Receiver Independent Exchange Format (RINEX) format [3]) to scientists, operators and downstream services across Europe.

The API in question has been in operation, in various forms, for several years. Over that period the code base accumulated significant technical debt: methods of five thousand lines, files of more than thirty thousand lines, duplicated logic copy-pasted across resources, and a long tail of patches added by successive developers and contributors to handle individual edge cases. A complete refactor of the API was therefore initiated, in which large parts of the

application were rewritten from scratch on top of Jakarta EE 11 and the Payara Platform [4, 5], while a small core of well tested logic was carried over from the previous version.

A refactor of this magnitude poses a very specific risk: any seemingly innocuous change in a query, a response field or a validation rule can produce subtle, hard to detect regressions that only manifest when a particular client, a particular station or a particular file is requested. Until this project, validation of those changes was carried out manually, a developer would deploy the new version, run a few requests by hand, and compare the resulting payloads against the previous one. This is precisely the situation against which this work is positioned.

1.2 Motivation

The motivation for this project is the gap between the pace at which the new API is being developed and the rate at which a human can reasonably validate that each new version behaves exactly like the previous one on the cases that already worked. Manual testing of a REST API that returns thousands of records, in several formats (JavaScript Object Notation (JSON), eXtensible Markup Language (XML), Comma-Separated Values (CSV)) and through dozens of query combinations, is both error prone and slow. Worse still, it does not scale to the cadence of a continuous refactor where commits land several times a day.

The need is twofold. On the one hand, the laboratory wanted an *automated* mechanism to compare every new build of the API against the previous one and flag behavioural divergences early, before they reach the production deployment. On the other hand, the laboratory also wanted to bring this mechanism inside a real Continuous Integration (CI)/Continuous Delivery (CD) pipeline running on infrastructure that the laboratory itself controls: a self-managed GitLab instance, paired with one or more GitLab Runners hosted inside the laboratory's network. The combination of the two is what turns a one-off testing script into a sustainable practice.

In addition to the immediate value for the EPOS API, the exercise of designing, prototyping and integrating such a framework was also seen as a vehicle for the student to gain hands-on experience with modern Jakarta EE features, with CI/CD pipelines, with self-hosted infrastructure, and with the practice of regression testing against a moving baseline, skills that are directly relevant beyond the scope of this particular project.

1.3 Objectives

In line with the project proposal, the work pursues four objectives: to design and implement an automated regression testing framework for the Jakarta EE REST API on the Payara Platform, with multi-format response comparison (JSON, XML, CSV); to integrate it into a self-managed GitLab CI/CD pipeline running on VPN-accessible runners; to validate the approach on two prototype applications before applying it to the production API; and to produce structured per-commit comparisons, both at the level of internal logic and at the level of Hypertext Transfer Protocol (HTTP) responses, in a form consumable by both human developers and the CI system.

The main contributions are a reusable three-stage GitLab pipeline template for Jakarta EE applications, a code-level differential test harness (`DiffHarness.java`) that loads two builds in isolated class loaders and compares their outputs, a complementary HTTP-level harness (`HttpDiff.java`) that boots one Payara Micro per available Web Application Archive (WAR) and performs a three-way comparison between HEAD, the immediate parent commit HEAD~1 and a fixed reference commit identified by the `DIFF_FIXED_BASELINE` CI/CD variable, and two prototype Jakarta EE 11 applications retained as didactic examples for future students.

1.4 Organization of the Document

The remaining chapters of this document are organised as follows. **Chapter 2** reviews the state of the art in test frameworks, CI/CD systems, and differential testing. **Chapter 3** describes the concrete technology stack adopted during the project. **Chapter 4** explains the iterative methodology (two prototypes followed by the production application) and the overall architecture of the testing framework. **Chapter 5** applies the framework to the production API (`api_v4.0`) and walks through the pipeline, both differential harnesses and the reports they produce. **Chapter 6** summarises the results, reflects on what was learned and proposes directions for future work.

1.5 Repository Access

The production API (`api_v4.0`) is hosted at https://gitlab.com/segalubi/api_v4.0 under the `segalubi` group with access restricted to laboratory members. The two prototypes are public in the author's personal namespace: <https://gitlab.com/PatalJunior/java-ee-app> and <https://gitlab.com/PatalJunior/syncspace-jakarta-app>. Each

repository contains a `README.md` with build instructions and (where applicable) the `.gitlab-ci.yml` discussed in Chapters 4 and 5.

Chapter

2

State of the Art

2.1 Introduction

This chapter reviews the body of practice and tooling relevant to the problem addressed by this project: how to automatically validate a Jakarta EE REST API, build after build, against a moving reference. It is organised around three axes. Section 2.2 surveys the main families of test frameworks used in the Java world, from low-level unit testing to in-container and HTTP-level testing. Section 2.3 discusses the CI/CD platforms that are commonly used to orchestrate those tests, with particular focus on GitLab, which is the platform used by the SEGAL laboratory and which directly constrains the design described later in the report. Section 2.2.1 reviews differential and regression testing approaches that influenced the design of the harnesses presented in Chapter 5. Section 2.4 closes the chapter.

2.2 Test Frameworks for the Java Ecosystem

The Java test landscape spans four layers, each combined by modern projects: unit testing (**JUnit 5** [6], the de facto standard, chosen here for its first-class Maven Surefire/Failsafe support and native JUnit XML output understood by GitLab; **TestNG** [7] remains a legacy alternative), integration testing (Maven Failsafe runs `*IT.java` during the `verify` phase; **Testcontainers** [8] stands up real infrastructure in Docker containers per test run, but was not adopted because the production database fixture is already reachable over the laboratory VPN), container testing (**Arquillian** [9], used in the first prototype to validate the CI/CD pipeline through its *micro-deployment* pattern, but rejected for the production API which is too large for per-test redeployments

and needs side-by-side comparison rather than isolated deployment), and API-level testing (**REST Assured** [10] provides a fluent Java Domain Specific Language (DSL); **Postman** with **Newman** [11, 12] can replay JSON collections in CI). For the present project, both were ultimately rejected in favour of plain `java.net.http.HttpClient`, which maps more naturally onto a differential workflow where the same request is sent to multiple ports and the responses are diffed pairwise.

2.2.1 Differential Testing

The directly relevant family of approaches is *differential testing*: instead of asserting a specific expected response, the candidate is compared against a reference implementation (or version) on a shared input grid. The canonical example in the service world is **Diffy** [13], a proxy released by Twitter Engineering that sits in front of a candidate and a primary service and replays each incoming request against both. This is exactly the situation faced by the SEGAL laboratory during the refactor of its API. *Snapshot testing*, popularised in the JavaScript world by Jest, was also considered but scales poorly when the output depends on a live database, as is the case here. The approach taken in this project is therefore closer to Diffy in spirit: rather than freezing a single reference, the baseline is always the parent commit of the build under test (plus an optional long-term fixed reference), which keeps the comparison meaningful across long refactor branches without requiring a human to maintain fixtures.

2.3 Continuous Integration and Continuous Delivery

2.3.1 Foundations

The notion of CI/CD as articulated by Humble and Farley [14] is the practice of integrating code into a shared mainline frequently, typically several times a day, and of keeping that mainline in a state from which a deployable artifact can be produced at any time, through a fully automated pipeline. The pipeline is canonically organised into a small number of stages, build, test, deploy, with strict gating between them: no stage runs unless the previous one has succeeded, and a failure stops the pipeline and surfaces a clear signal to the team.

The same publication popularised the idea of the *deployment pipeline* as the single source of truth about a build's quality. In a mature setup, the artifact

produced by the build stage is the exact same artifact that flows through the test stages and eventually to deployment, ensuring there is no drift between what was tested and what was shipped. The same principle is applied in this project: the WAR built once in the first stage of the pipeline is consumed verbatim by the testing stages later on.

2.3.2 Platforms

Three platforms dominate the modern CI/CD space and are worth comparing in the context of the laboratory's choice.

Jenkins [15] is the long-standing, plugin-based, self-hosted automation server. Its strengths are flexibility and breadth of plugin coverage; its main drawbacks are that pipelines are historically described in Groovy (with the related learning curve) and that the maintenance burden of the master/agent topology is non-trivial.

GitHub Actions [16] is a hosted CI/CD service that integrates tightly with repositories hosted on GitHub. Its pipelines are described in *YAML Ain't Markup Language* (YAML) and its ecosystem of reusable *actions* is large. It is a very strong default for open source projects, but it cannot be self-hosted as an end-to-end platform; the control plane runs on GitHub's infrastructure.

GitLab CI/CD [17] is the CI system integrated into GitLab. Pipelines are described in a single `.gitlab-ci.yml` file at the root of the repository; runners are separate processes that pick up jobs from the GitLab server, and can be *shared* (provided by gitlab.com) or *specific* [18], registered against a particular project or group. The feature that matters most for this project is that GitLab itself can be *self-managed* [19]: the entire control plane can be installed inside the laboratory's network, and combined with one or more locally-hosted runners can be used to build, test and deploy applications that depend on internal resources such as the laboratory's GNSS database, without exposing those resources to the public Internet.

That last property is the deciding factor. The SEGAL laboratory operates a self-managed GitLab instance and a GitLab Runner host provisioned specifically for the EPOS API; that runner is inside the laboratory's network and can reach the production database directly. The choice of GitLab CI/CD as the orchestration platform for this project is therefore not an abstract preference but a consequence of the operating environment.

2.4 Conclusion

The state of the practice in Java API testing offers a rich catalogue of tools at each layer of the stack, and a small number of mature CI/CD platforms in which to orchestrate them. From this catalogue, the project adopts JUnit 5 as the unit/integration test framework, plain `java.net.http.HttpClient` as the HTTP client for the differential harness, and GitLab CI as the orchestration platform, the last choice being effectively mandated by the laboratory's infrastructure. Differential testing, as embodied by tools like Diffy, provides the conceptual model on top of which the bespoke harnesses described in Chapter 5 are built. The next chapter details the concrete stack of technologies on which the implementation rests.

Chapter

3

Technologies and Tools Used

3.1 Introduction

This chapter describes the concrete set of technologies and tools on which the work reported in this document is built. The set was not chosen in a single decision, but converged over the course of the project through a mixture of constraints (what the production API already uses, what the laboratory’s network allows, what the proposal explicitly mandates), comparisons (between equivalent alternatives) and pragmatic preference. The chapter is structured by layer of the stack: API contracts and serialization formats in Section 3.2, the Java/Jakarta ecosystem in Section 3.3, persistence and Structured Query Language (SQL) databases in Section 3.4, and servers, tooling and network in Section 3.5.

3.2 APIs, Serialization Formats and Specification

3.2.1 REST and the Multi-format Response

All endpoints exposed by the applications discussed in this report follow the REST architectural style as described by Fielding [20]: resources are identified by stable Uniform Resource Locators (URLs), manipulated through a small set of HTTP verbs (with GET dominating for read-only GNSS metadata), and the API is stateless on the server side. There is no session affinity, which is a property that the differential test harness described in Chapter 5 relies on heavily: the same request can be replayed against two independent server processes and the responses can be compared directly.

The production API (`api_v4.0`) is unusual in that it serves the *same* resource in several serialization formats, selected by a path parameter at the end of the URL. The supported formats are JSON (default), XML (produced through Jackson's XML data format module), CSV (produced through a custom writer) and, for the file metadata endpoints, an executable shell *script* that downloads the corresponding files. This multi-format output was inherited from the legacy API, many of its clients are scientific scripts that consume CSV or JSON directly, and was preserved in the refactor; it is also the reason the HTTP-level differential harness has to compare not only headers and status codes but also the body, format by format. Figure 3.1 sketches the content negotiation pattern that turns a single resource method into four wire formats.

3.2.2 JSON and XML

JSON is the default serialization and the format on which most of the test fixtures of this project rely. The Jakarta REST implementation provided by Payara takes care of object serialization through Jackson, with negligible per-request overhead. XML is provided by the `jackson-dataformat-xml` module, configured to preserve the same Java models that produce JSON. This dual configuration is valuable because it removes a whole class of *schema drift* bugs: as long as the Java model is correct, both JSON and XML outputs are consistent with each other.

3.2.3 CSV

CSV is the format favoured by historical clients of the API and is produced through a small set of writer classes that flatten the entity tree into rows. The differential harness handles CSV exactly the same way it handles JSON and XML: the textual body of the response is treated as an opaque blob that must match, byte for byte, against the baseline.

3.2.4 OpenAPI Specification (OpenAPI) and Swagger User Interface (UI)

The contract of the production API is described in an OpenAPI 3.1.0 [21] document (`openapi.json`) that ships inside the WAR. The same document is served to a Swagger UI [22] instance hosted at `/swagger.html`, which provides an interactive, in-browser playground for the endpoints. Two practical benefits arise from this setup. First, the document is the single source of truth

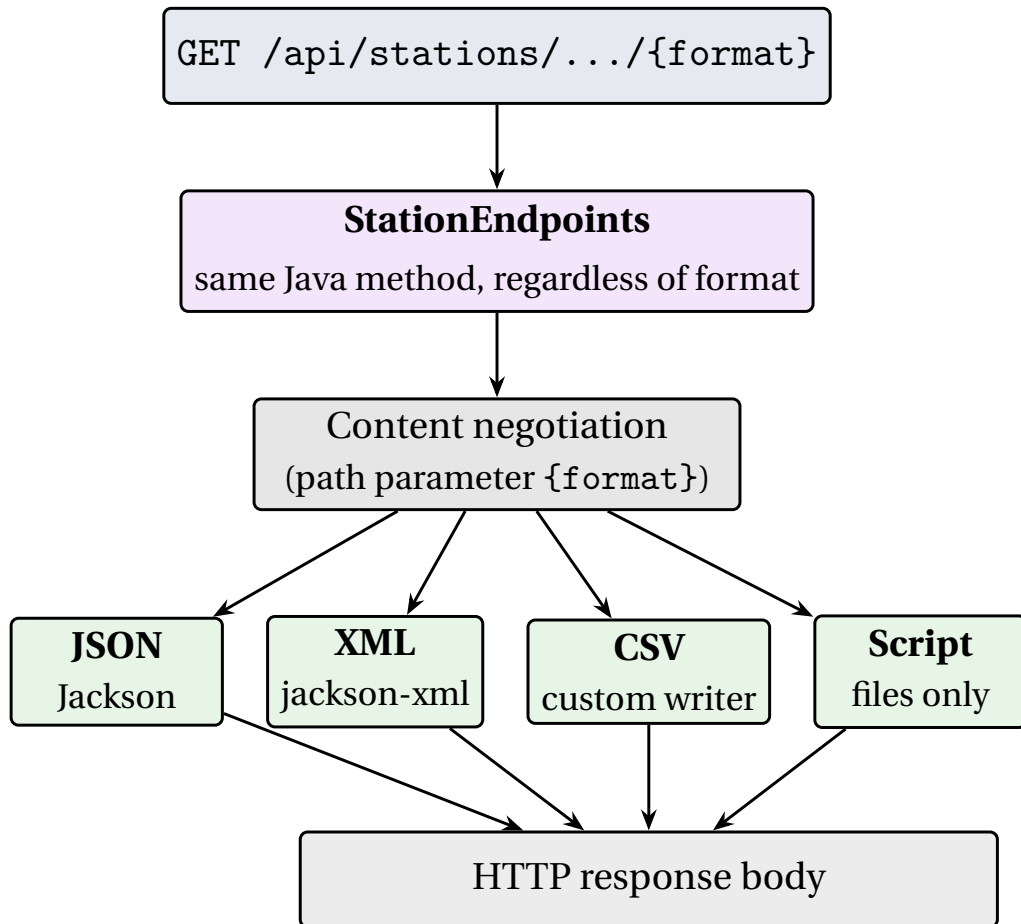


Figure 3.1: Multi-format content negotiation in `api_v4.0`. The same Java resource method is reused across four serializations, selected by a path parameter, and the differential harness must compare the body of each format separately.

shared with downstream consumers of the API, in particular the EPOS portal. Second, the same document can be used to drive contract tests: a future evolution of the test framework, briefly discussed in Chapter 6, is to derive the HTTP differential test cases automatically from the OpenAPI specification instead of maintaining them in a separate JSON file.

3.3 Java and Jakarta EE

The applications target Java SE 21 [23] (the production API and the second prototype) and Java SE 17 (the first prototype, kept compatible with the

maven:3.9.6-eclipse-temurin-21 CI image). The choice of Java was effectively imposed by the existing code base; record types are used liberally for request/response Plain Old Java Objects (POJOs) and the new switch expression syntax replaces several long if/else chains from the legacy API.

Jakarta EE 11 [4] provides the platform layer. The renaming from Java EE in 2018 had already been absorbed by the time this project started; the current code base targets the `jakarta.*` package namespace. The specifications actively used across the three applications are: Jakarta REST 4.0 (Jakarta RESTful Web Services (JAX-RS)) [24] for endpoints, Jakarta *Contexts and Dependency Injection* (CDI) 4.1 for injection, Jakarta Persistence 3.2 (Jakarta Persistence API (JPA)) [25] for Object-Relational Mapping (ORM), Jakarta Bean Validation 3.1 for declarative constraints, Jakarta JSON Binding 3.0 for serialization, and Eclipse MicroProfile [26] 2.1 JSON Web Token (JWT) (used by the second prototype) plus Health 4.0 for probes. Spring Boot [27] was considered but rejected: the existing code base is already on Jakarta EE on top of an existing Payara installation, and a parallel migration would have brought no proportional benefit.

3.4 SQL and the Database Layer

The persistence model of the production API is heavily relational: GNSS stations, networks, agencies, countries, file types, observation summaries and embargo windows are all linked through foreign keys, and most endpoints execute several joins per request. SQL [28] is therefore the natural query language, and even where JPA's object-oriented abstractions (Section 3.4.1) are used, what reaches the database is plain SQL. The laboratory standardised on **PostgreSQL** [28] for the EPOS production database; the choice over **MySQL** [29] was driven mainly by PostgreSQL's adherence to the SQL standard (advanced JOIN types, common table expressions, the `numeric` type for high-precision station coordinates) and by its rich set of index types (B-tree, BRIN, GiST). The PostGIS extension is not in use today (Section 3.4.2) but remains a realistic future addition. MySQL was nevertheless used in the docker-compose configuration of the second prototype (Section 4.3) to validate that the application-level code is not tied to a specific dialect.

3.4.1 JPA, Hibernate and the Persistence Unit

JPA [25] is the Jakarta specification for object-relational mapping in Java. The production API uses Hibernate [30] as the JPA provider, configured through a `persistence.xml` file. The persistence unit is named `MeuPU` and is declared

as `RESOURCE_LOCAL`, which means that transactions are managed explicitly by application code rather than by the container, a deliberate choice that keeps the differential test harness in full control of when transactions begin and end.

Database connections are pooled through **HikariCP** [31] with a small, conservative pool (2–15 connections, 20-second connect timeout, 30-second idle, 5-second validation), tuned for the read-mostly workload of the API and for the relatively low concurrency observed in production.

3.4.2 The EPOS Database in Practice

The production database lives on a PostgreSQL 16 instance reachable from the laboratory network at 192.168.168.30:5432, and the same schema serves two fixtures: `DB_3.0_J`, a small but representative production subset used by the correctness diffs, and `DB_3.0_F`, the full dataset used for volume-oriented and performance experiments. The schema contains forty-two tables and around fifty foreign-key constraints, grouped into a core domain (station, network, agency, country, `rinex_file`, `highrate_rinex_file`, `file_type`), the junction tables that materialise its many-to-many relationships, a family of seven quality-control summary tables, and a handful of auxiliary tables (embargoes, documents, licenses, infrastructure). Station coordinates are plain numeric; spatial logic is currently performed in Java by the Geometry package rather than by PostGIS, which is not installed (the only non-default extension is `plpgsql`). The twenty-five existing indexes are mostly B-trees on foreign-key columns, with `fillfactor=90` on the read-heavy quality-control tables. The next planned optimisation, and the reason the larger F fixture was provisioned, is the introduction of BRIN indexes on the date columns of `rinex_file`: RINEX files are written in approximate date order, so the on-disk layout already correlates with the indexed column, which is the precondition for a small and fast BRIN index. The differential framework introduced in Chapter 5 is used to validate that this optimisation does not change observable behaviour against the F fixture.

3.5 Servers, Tooling and Network

Payara Server [5] is the Jakarta EE application server adopted by the laboratory; the production API is deployed onto an existing Payara installation. Its companion product, **Payara Micro**, is a single self-contained Java Archive (JAR) that launches a deployable WAR from the command line and is the engine on which the HTTP-level differential harness relies (the harness boots one Payara Micro per WAR on different ports and replays the same requests

against each). Apache Tomcat [32] was considered for the prototypes but rejected because it does not ship a full Jakarta EE 11 runtime out of the box.

All three applications are built with **Apache Maven**; the production `pom.xml` configures, among others, the `maven-war-plugin` (which produces the deployable `ROOT.war`), `maven-failsafe-plugin` (integration tests named `*IT.java` with **JUnit!** (**JUnit!**) XML output) and `maven-dependency-plugin` (which stages the Payara Micro JAR into `target/payara-micro/` so the HTTP harness can spawn it as a subprocess).

GitLab [17] provides both the source-control front end and the CI/CD platform. The GitLab server is a self-managed installation [19] running on a Virtual Machine (VM) inside the laboratory; the GitLab Runner [18] is a separate process on another VM (tagged `ubi-runner-.31`) that picks up jobs and executes them inside Docker containers. That separation matters because the runner is the machine that needs network access to the laboratory's database, not the control plane. **Git** [33] itself is used non-trivially in the pipeline: `git worktree` is what materialises each baseline build alongside the HEAD build on the runner's filesystem, without disturbing the working copy (Section 5.2). All three repositories are hosted on `gitlab.com`, the production API under the `segalubi` group, the two prototypes under the personal namespace of the author.

Docker [34] appears in two roles. CI jobs run inside the official `maven:3.9.6-eclipse-temurin-21` image, which guarantees a clean and reproducible toolchain regardless of the runner host. The two prototypes additionally ship a `docker-compose.yaml` [35] for local development (PostgreSQL plus, in the first prototype, a pair of Payara Server containers for the dual-node Arquillian tests). The production API is intentionally *not* containerised: the deployment model is to copy the WAR onto an existing Payara installation, and an image would add a moving part with no benefit (this is revisited in Chapter 6).

Finally, the production database is reachable only inside the laboratory's network on `192.168.168.30:5432`, which is why the runner is pinned by tag and why developers operating from outside connect through the laboratory's VPN, maintained by the SEGAL staff. Alternatives such as WireGuard [36] were considered as lighter-weight options for one-off remote sessions but were not adopted.

3.6 Conclusion

The technology stack on which this project is built is in large part imposed by its environment: Jakarta EE on Payara, PostgreSQL on the internal network,

GitLab as the self-managed CI platform. Where there was room for choice, between Jakarta EE and Spring Boot, between Payara Server and Apache Tomcat, between PostgreSQL and MySQL, between JUnit 5 and TestNG, the decisions documented in this chapter were guided by a preference for fewer moving parts, by the alignment with what the laboratory already runs in production and by the desire to keep the differential testing harness as simple as possible. The next chapter describes the iterative methodology by which these technologies were combined into a working solution.

Chapter

4

Methodology and Architecture

4.1 The Legacy API and the Need for a Safety Net

The starting point of the work is a code base that had grown organically for several years and that nobody fully understood any more: the previous version of the EPOS API contained Java methods of more than five thousand lines, source files of more than thirty thousand lines, and many subtle behavioural quirks introduced as patches by successive developers in response to specific bug reports. The corollary was that no member of the laboratory could realistically guarantee, for an arbitrary change, that no client of the API would be silently broken. The full rewrite on top of Jakarta EE 11 made the internal structure much cleaner (short methods, well-bounded JAX-RS resources, clear separation between parsing, validation, query construction and serialization) but opened a window of risk: at any given commit during the refactor, an endpoint may produce subtly different output from the version deployed in production. The role of this project is to build the safety net that turns those subtle differences into a visible signal in the CI pipeline.

4.2 Iterative Methodology

The work was organised into three iterations, each carried out in its own dedicated repository. This separation served two purposes. First, it kept each iteration small and focused: the first prototype is deliberately stripped down to two trivial endpoints, so that the entire complexity is in the CI configuration and not in the application; the second prototype is the opposite, focusing on application code and leaving CI aside; the production API then brings the two

strands back together. Second, the prototypes have value in their own right as didactic artifacts that future students of the laboratory can read and re-use.

4.3 Prototype Applications

The first prototype, `java-ee-app`, validated the CI/CD infrastructure end-to-end before any of the production code base was touched: it is a minimal Maven WAR exposing two trivial JAX-RS endpoints, with a `.gitlab-ci.yml` that builds the artifact and runs Arquillian integration tests against two managed Payara Server containers declared as *services* in the job. Three patterns from that pipeline were carried over verbatim to the production one: the pinned base image with the `ubi-runner` tag, the global Maven repository cache, and the reports: `junit:` stanza that makes GitLab understand assertion failures as failed pipeline steps.

The second prototype, `syncspace-jakarta-app`, focused on the opposite concern: a complete Jakarta EE 11 application with no CI pipeline, exercising the specifications listed in Section 3.3 on a small room-booking domain (entities `User`, `Room`, `Booking`) protected by MicroProfile JWT [26]. JAX-RS [24] declares the resources, CDI wires the services, JPA [25] maps the entities, Bean Validation enforces a custom `@ValidDateRange` constraint, and a focused set of four `ExceptionMappers` (`BusinessValidationMapper`, `ValidationExceptionMapper`, `JsonProcessingExceptionMapper`, `GlobalExceptionMapper`) translates domain and framework exceptions into structured JSON errors without leaking stack traces. The end-to-end exception flow is summarised in Figure 4.1.

Two patterns from `syncspace-jakarta-app` were adopted by the production refactor: the use of a small set of focused `ExceptionMappers` (which keeps HTTP status codes meaningful and gives the differential harness a stable signal to assert on) and the use of record-based POJOs for request and response bodies (which simplifies diffing). The remaining features, in particular the full JWT flow, were retained in the prototype as a self-contained reference example for future students.

4.4 Architecture of the Differential Testing Framework

The architecture of the final testing framework, applied to the production API, is summarised in Figure 4.2. It is made of three concentric layers: the GitLab pipeline, the harness processes, and the application instances themselves.

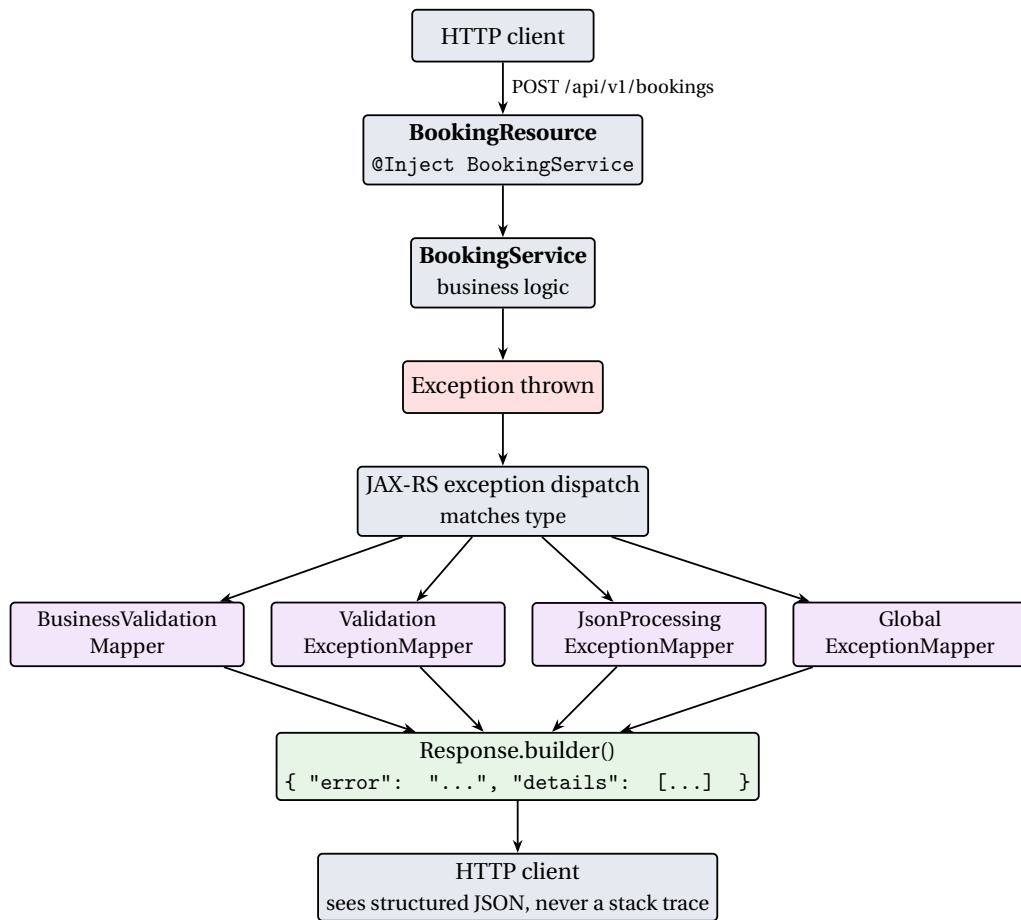


Figure 4.1: The CDI + JAX-RS exception-handling flow used by syncspace-jakarta-app and partly inherited by api_v4.0. Any exception thrown inside a resource or service bean is intercepted by the most specific registered ExceptionMapper, which produces a stable JSON error body.

A push by the developer triggers a pipeline on the runner tagged ubi-runner-.31, which goes through four jobs across three stages: build-job produces the HEAD WAR, integration-test-job runs the regular JUnit integration tests, validate-diff-job reconstructs both baseline WARs via Git worktrees and runs the code-level diff harness against each in turn, and http-diff-job boots all three Payara Micro instances on different ports, replays http-diff-cases.json against each and emits per-baseline diff reports. All reports are picked up by GitLab as both JUnit results and raw artifacts.

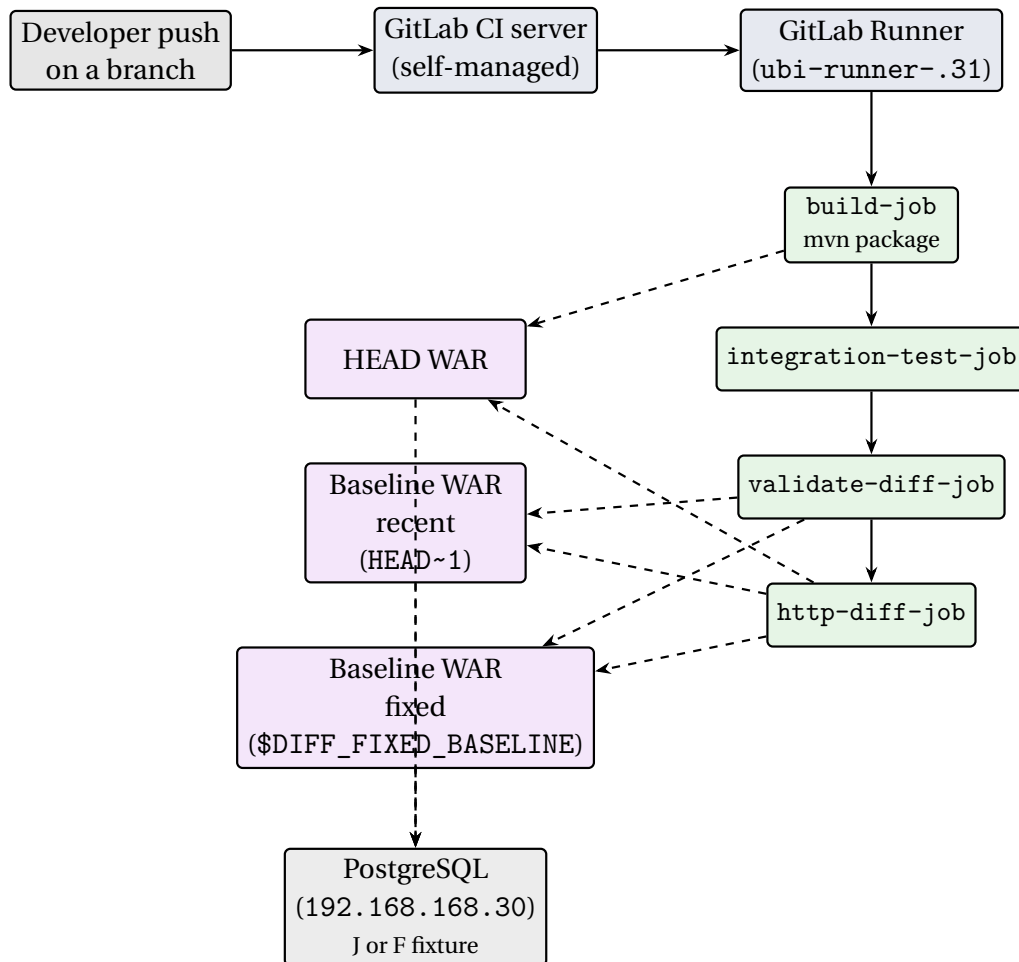


Figure 4.2: Overall architecture of the differential testing framework. Solid arrows denote control flow inside the pipeline; dashed arrows denote artifact or data access. Both diff jobs run a three-way comparison between the HEAD build and two baselines, the immediate parent commit and a fixed reference commit, all pointed at the same database fixture.

The two prototypes traded a longer time-to-result for a sharper separation of concerns between CI/CD infrastructure and Jakarta EE feature usage; the next chapter applies the resulting framework to the production API.

Chapter

5

Use Case: EPOS API

This chapter applies the framework of Chapter 4 to the production API, `api_v4.0`. The EPOS itself [1] is a pan-European research infrastructure in solid Earth science, organised into Thematic Core Services for seismology, geomagnetic observations, GNSS data and products [2], and so on; the SEGAL laboratory of UBI contributes to the GNSS service by operating a node that hosts station metadata and RINEX [3] observation files, and `api_v4.0` is the public interface of that node. Three properties of the domain shape the test framework: the entities are relatively static (stable baselines are easy to obtain), the typical client request returns a long list of records (the comparison algorithm must scale to long diffs), and the same information is exposed in several serialization formats (the surface area to test multiplies but the underlying logic does not).

5.1 Architecture of `api_v4.0`

The application is a Maven WAR project targeting Java SE 21 and Jakarta EE 11, deployed onto Payara Server 6 with `finalName=ROOT` so that it serves the root context. Its source tree is organised into four top-level packages that form a classical layered architecture (Figure 5.1): `Glass` provides the REST layer (JAX-RS resources `StationEndpoints`, `FilesEndpoints`, `ListEndpoints` and `Start`, plus parsers, format writers and the validation/shape helpers); `Geometry` and the helper classes `Validate.java` and `Shape.java` form a thin business layer; `DB_Tables` holds the forty JPA entities that model the relational schema (stations, networks, agencies, file types, observation summaries, embargoes); and `Config` (`Loader.java`) is the cross-cutting concern that handles bootstrap.

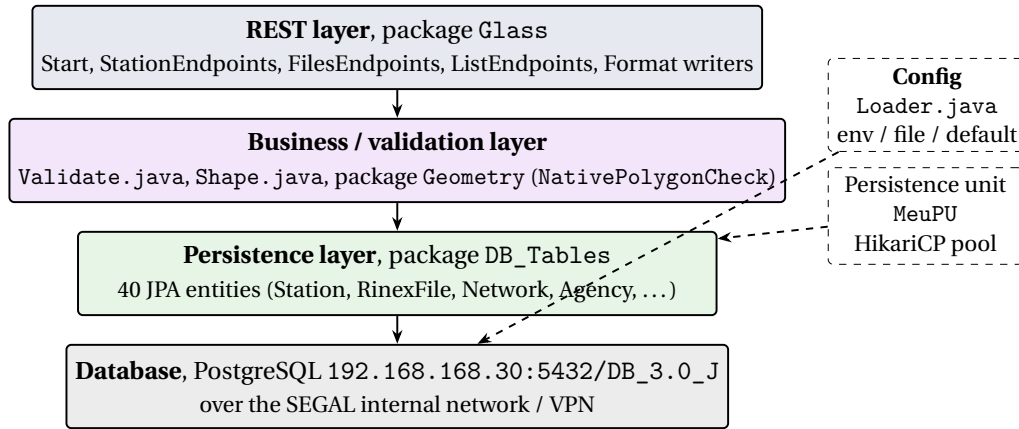


Figure 5.1: Layered architecture of `api_v4.0`. Solid arrows denote ordinary call/data flow; dashed arrows denote configuration and infrastructure binding established at deployment time.

5.1.1 Endpoints and Output Formats

The REST surface is declared in four classes inside the Glass package. `Start.java` carries the `@ApplicationPath("/api")` annotation and registers a `PingResource` for the trivial `/api/ping` liveness check. The three substantive resources are `StationEndpoints` (`/api/stations`, seven endpoints for station metadata indexed by identifier, location, agency, network, equipment or coordinates), `FilesEndpoints` (`/api/files`, four endpoints for RINEX file metadata filtered by agency/antenna/bounding box/date, by marker, the high-rate variant, and an aggregated combination), and `ListEndpoints` (`/api/list`, a parametric endpoint returning the valid values of a given filter). Every endpoint accepts a `{format}` path parameter and emits JSON, XML, CSV or (for the file endpoints) a downloadable shell script; the same OpenAPI 3.1.0 document is served at `/swagger.html` through a Swagger UI hosted inside the WAR.

5.1.2 Persistence

The persistence unit, declared in `src/main/resources/META-INF/persistence.xml`, is named `MeuPU` (inherited from the legacy code). It is configured as `RESOURCE_LOCAL`, with Hibernate as the JPA provider and a connection pool managed by HikariCP. Database connection parameters, host, port, database name, user, password, are resolved at startup by `Config/Loader.java`, with the following precedence: environment variables, then a `Config.config` file at the root of the deploy-

ment, then hard-coded defaults pointing at the laboratory’s database (192.168.168.30:5432).

The laboratory maintains two database fixtures with the same schema but different volumes of data:

- DB_3.0_J hosts a small but representative production subset, on the order of ten stations, eighty-six networks, two hundred and thirty agencies and roughly one hundred and forty-six thousand `rinx_file` rows at the time of writing. This is the fixture used by the correctness diff runs, because it is small enough to allow per-case comparison of full response bodies without overwhelming the report.
- DB_3.0_F hosts the full dataset and is used by the volume-oriented diff runs, where the goal is not byte-for-byte equality but the detection of changes that would only be apparent at scale, for example, an inadvertent N+1 query introduced by a refactor, or a query plan regression visible only on a large table.

The precedence rules of `Loader.java` are what allow the `http-diff-job` to switch the running WARs between the two fixtures without rebuilding the artifact: the job sets the appropriate environment variables in its `variables` block and the application picks them up at boot.

The shape of the schema is discussed in Section 3.4.2; a simplified view of the central entities is given in Figure 5.2.

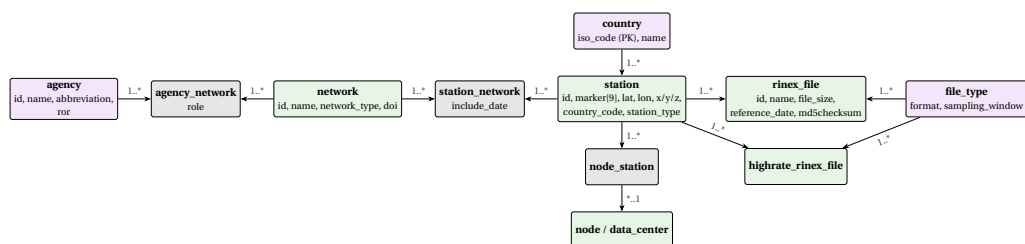


Figure 5.2: Simplified view of the core EPOS data model, with representative columns as they appear in the live PostgreSQL schema. Green entities carry domain data; purple entities are controlled-vocabulary lookups (note that `country.iso_code` is the natural primary key, not a surrogate id); grey entities are junction tables that materialise the many-to-many relationships of the model. Arrows point from the “one” side to the “many” side of the relationship.

5.2 The CI/CD Pipeline

The pipeline of `api_v4.0` extends the patterns validated on the first prototype with two additional jobs in a new `diff` stage. Each `diff` job compares the current build against *two* baselines, in what amounts to a three-way comparison: the immediate parent commit `HEAD~1` (which catches regressions introduced by the commit currently being pushed) and a *fixed* reference commit identified by a SHA stored in the `DIFF_FIXED_BASELINE` GitLab CI/CD variable (which catches the slow drift that accumulates over many commits along a long-running refactor branch). The key parts of the `.gitlab-ci.yml` are shown, condensed, in Listing 5.1.

```
default:
  image: maven:3.9.6-eclipse-temurin-21
  tags: [ ubi-runner-.31 ]
cache: { key: global-maven-cache, paths: [ .m2/repository ] }
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/
    repository"
  GIT_DEPTH: "20"
stages: [ build, test, diff ]

build-job:
  stage: build
  script: [ mvn clean package -DskipTests ]
  artifacts: { paths: [ target/*.war ] }

integration-test-job:
  stage: test
  script: [ mvn verify ]
  artifacts:
    when: always
    reports: { junit: [ "target/failsafe-reports/TEST-*.xml" ] }

validate-diff-job:
  stage: diff
  needs: [ { job: build-job, artifacts: false } ]
  variables: { DIFF_FIXED_BASELINE: "" } # commit SHA, CI/
    CD var
  script:
    - |
      run() { # $1=label, $2=sha
        [ -z "$2" ] && return 0
        WORK=/tmp/$1-baseline-build
        git worktree add "$WORK" "$2"
        ( cd "$WORK" && mvn -q -o package -DskipTests )
```

```

        export BASELINE_WAR="$WORK/target/ROOT.war"
        DIFF_LABEL="$1"
        export DIFF_REPORT_DIR="target/diff-report/$1"
        mkdir -p "$DIFF_REPORT_DIR"
        mvn test -Dtest=DiffHarness -DfailIfNoTests=false
    }
    run recent "$(git rev-parse --verify HEAD~1 2>/dev/
        null)"
    run fixed "$DIFF_FIXED_BASELINE"
artifacts:
  when: always
  paths: [ target/diff-report/ ]
  reports: { junit: [ "target/surefire-reports/TEST-Diff.
        DiffHarness.xml" ] }

http-diff-job:
  stage: diff
  needs: [ { job: build-job, artifacts: true } ]
  variables:
    DATABASE_URL: "postgres://postgres:postgres@192
        .168.168.30:5432/DB_3.0_J"
    JAKARTA_SCHEMA_ACTION: "none"
    DIFF_FIXED_BASELINE: ""
  script:
    - |
      # build baselines if available, then let HttpDiff
      # orchestrate
      # spawning Payara Micros on ports 8080/8081/8082.
      [ -n "$(git rev-parse --verify HEAD~1 2>/dev/null)" ]
      && \
        export BASELINE_RECENT_WAR=$(build recent HEAD~1)
      [ -n "$DIFF_FIXED_BASELINE" ] && \
        export BASELINE_FIXED_WAR=$(build fixed "
            $DIFF_FIXED_BASELINE")
      mvn test -Dtest=HttpDiff -DfailIfNoTests=false
  artifacts:
    when: always
    paths:
      - target/http-diff-report/
    reports:
      junit:
        - "target/surefire-reports/TEST-Diff.HttpDiff.xml"
    expire_in: 4 weeks

```

Code Listing 5.1: Condensed view of the production `.gitlab-ci.yml`, three stages and four jobs.

Three details of the listing are worth highlighting. The *recent* baseline is `HEAD~1`, which keeps the per-commit comparison focused on the change

just introduced; the *fixed* baseline is a commit Identifier (ID) stored in `DIFF_FIXED_BASELINE` (typically pointing at the last commit known to have shipped to the EPOS portal) and acts as a long-term reference against which the cumulative drift of the branch is measured. Each baseline build is materialised through `git worktree` rather than `git checkout`, so the working copy is never disturbed and the harness can freely compare files from both checkouts in parallel. Finally, each baseline is optional and treated independently: if `HEAD~1` does not exist (first commit of the repository) or `DIFF_FIXED_BASELINE` is unset (start of a refactor branch), the corresponding sub-run is skipped. A pipeline can therefore have zero, one or two diff sub-runs depending on the situation, and never fails because a baseline is simply unavailable.

5.3 The Code-Level Diff: `DiffHarness.java`

`DiffHarness.java` is a JUnit 5 test that compares two versions of a specific Java class on a shared input grid, in process. Its core idea is to load the baseline and the HEAD WARs through two independent `URLClassLoader` instances, to reflect into the class under test on both sides and to invoke each method of interest with the same arguments. Any difference between the two returned values, thrown exceptions included, is recorded as a row in the diff report.

The pipeline invokes the harness once per baseline (recent and fixed; see Section 5.2). The exported environment variables `BASELINE_WAR`, `DIFF_REPORT_DIR` and `DIFF_LABEL` tell the harness which WAR to load against HEAD and where to write its outputs. The on-disk layout is `target/diff-report/recent/` and `target/diff-report/fixed/`, so a glance at the artifact tree of a finished pipeline immediately distinguishes a per-commit regression from a long-term drift.

The harness focuses on two classes for which behaviour is critical and for which an entire endpoint cannot be exercised in isolation: `Validate`, which parses and validates client query parameters, and `Shape`, which evaluates geometric polygon queries against the bounding box of a request. Both classes are purely functional, take simple inputs (strings, lists of doubles) and return simple outputs (booleans, lists, error strings), which makes them well suited to a code-level diff.

The output of each sub-run is twofold. A **JUnit!** XML report (`TEST-Diff.DiffHarness.xml`, with the `DIFF_LABEL` appearing as a test-suite property) integrates with GitLab so that divergences appear as failed assertions on the pipeline page. In parallel, a richer CSV/JSON report under the per-baseline subdirectory records every input row, the value returned by the baseline, the

value returned by HEAD and an indicator of whether the two match. The CSV format makes the report easy to open in a spreadsheet for ad-hoc analysis.

5.4 The HTTP-Level Diff: `HttpDiff.java`

`HttpDiff.java` is the second harness, and the one that exercises the API end-to-end. Its execution proceeds in four phases, summarised graphically in Figure 5.3. Unlike the code-level harness, which is invoked twice by the pipeline, the HTTP harness runs once and internally boots *all* the available WARs in parallel (the HEAD WAR, the recent baseline if any, and the fixed baseline if any). It then issues every test case against every running instance and emits one diff report per baseline.

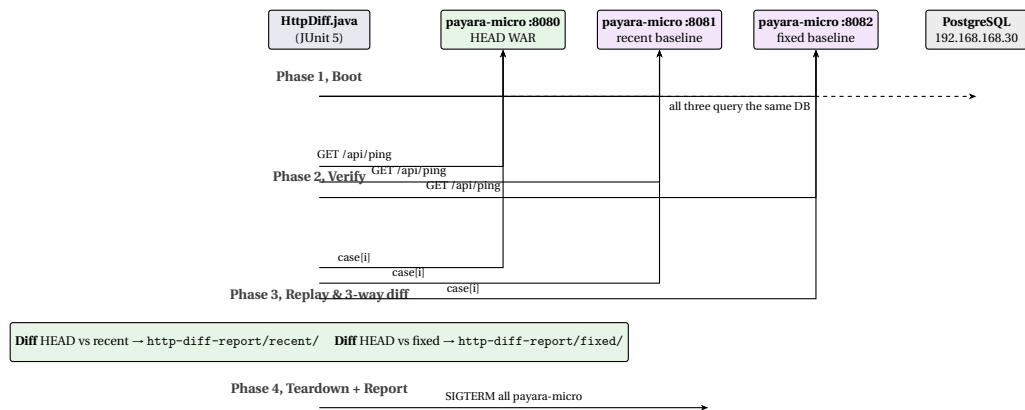


Figure 5.3: Phases of `HttpDiff.java`. The harness boots one Payara Micro per available WAR, the HEAD build and whichever of the recent/fixed baselines were provided, waits for all of them to be ready, fires every test case against every instance in parallel, then performs a three-way comparison and emits one diff report per baseline. All instances share the same real PostgreSQL fixture over the laboratory VPN.

The figure already covers the gist; a few implementation details are worth recording. The test cases live in `src/test/resources/http-diff-cases.json` as a JSON array of request descriptors (method, path, headers, optional body); the deploy-verification handshake added in commit 67a0da9 (“*prove deploy actually happened before counting it as ready*”) guards against spurious failures from requests landing on still-deploying containers. Bodies are compared as normalised strings (sorted JSON keys, trimmed whitespace) and the headers that vary by definition (Date, Server, Content-Length) are ignored. The two comparisons (HEAD

vs recent, HEAD vs fixed) are kept in separate report directories so that a per-commit regression and an accumulated drift remain distinguishable in the artifact tree.

5.5 Reports and Findings

Each pipeline run publishes the integration-test reports, the two diff reports (`recent/` and `fixed/` subdirectories of `target/{diff,http-diff}-report/`) and the JUnit XML files consumed by GitLab. Integration test results expire after one week; the diff reports are kept for four because they are the historical record of behavioural change across the refactor. During development the harnesses surfaced a handful of real regressions hidden inside seemingly cosmetic refactors. The most representative example is the extraction of latitude and longitude range bounds to named constants (commits `7c8ff37` and the subsequent `cc223ba` revert): the diff harness reported a silent behavioural change at the boundary that no human review of the diff would have caught by eye. Those two commits, in the author's view, are the best validation that the framework does what it was designed for. The next chapter draws the main conclusions of the project and outlines the items that, by choice or by constraint, were left out of its scope.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

The work described in this report addresses a concrete problem of the SEGAL laboratory: how to refactor a large, several-years-old Jakarta EE API without silently breaking the EPOS clients that depend on it. The answer is a three-stage CI/CD-driven testing framework that, for every commit, compares the new build against two baselines (the immediate parent commit and a fixed long-term reference) both at the level of internal Java classes and at the level of HTTP responses.

A few conclusions stand out. *Differential testing fits a refactor naturally*: asserting that the new build produces the same output as a known-good one sidesteps the brittle fixture problem and made the framework useful from the very first commit it ran on. *Two layers of comparison are better than one*: the code-level harness catches behavioural changes in focused helper classes with fast feedback, the HTTP-level harness catches integration regressions visible only against a live database, and neither one alone covers the spectrum the laboratory cares about. *A self-hosted pipeline is not optional here*: only a runner inside the laboratory's network can exercise the API end-to-end against the production database, so self-managing both the GitLab control plane and the GitLab Runner host followed from the data-locality constraint, not from a preference. *The two prototypes paid for themselves*: each answered a small, well-bounded set of questions in an environment where mistakes were cheap, sparing the production code base from them. *The framework reveals what a human review would miss*: the most representative example, discussed in Section 5.5, is the named-constants refactor of the latitude/longitude bounds that no amount of code review would plausibly

have caught at the laboratory's commit rate.

6.2 Future Work

Several extensions to the framework were deliberately left out of scope. The most immediate are the introduction of **BRIN** indexes on the date columns of `rinex_file` (and the companion *performance-regression subsystem* that would record wall-clock durations against the F fixture to justify them empirically), the **PostGIS migration** of the spatial helpers in the Geometry package (with the same differential framework used to verify byte-for-byte equivalence), and the **automatic generation of HTTP test cases** from the OpenAPI document shipped inside the WAR (so newly added endpoints are exercised by the diff from the moment they are merged). The framework would also benefit from a **schema diff** of the OpenAPI document itself across builds (to surface contract regressions distinct from behavioural ones), from a **snapshot test** layer for endpoints that should remain stable across releases (such as `/api/ping`), and from **containerization of the production deployment** to remove the residual gap between the WAR that was tested and the WAR that ships. Finally, the harness code itself could be **generalised into a small library** reusable by other Jakarta EE services of the laboratory, with the application-specific bits factored into a clean configuration interface; orthogonal to that, several **operational aspects** (transient-error retries, clean termination of orphan Payara Micro processes, automated provisioning of the runner host, a long-term dashboard on top of the diff reports) deserve more attention than the present project could afford.

Overall, the framework delivered by this project addresses the problem that motivated it, has already proved its value on a real regression caught during the refactor, and lays the groundwork for the laboratory to continue evolving the API with a much higher degree of confidence than was previously possible. The items listed in this section are natural next steps; none of them invalidates the design that was put in place.

Bibliography

- [1] EPOS ERIC. European Plate Observing System — EPOS, 2024. [Online] <https://www.epos-eu.org/>. Accessed June 2026.
- [2] EPOS GNSS Data and Products Thematic Core Service. EPOS GNSS Data Gateway, 2024. [Online] <https://gnss-epos.eu/>. Accessed June 2026.
- [3] Werner Gurtner and Lou Estey. RINEX — The Receiver Independent Exchange Format, Version 3.05, 2020. International GNSS Service (IGS), RINEX Working Group.
- [4] Eclipse Foundation. Jakarta EE 11 Platform Specification, 2024. [Online] <https://jakarta.ee/specifications/platform/11/>. Accessed June 2026.
- [5] Payara Services Ltd. Payara Platform Community Documentation, 2024. [Online] <https://docs.payara.fish/>. Accessed June 2026.
- [6] JUnit Team. JUnit 5 User Guide, 2024. [Online] <https://junit.org/junit5/docs/current/user-guide/>. Accessed June 2026.
- [7] Cédric Beust. TestNG documentation, 2024. [Online] <https://testng.org/>. Accessed June 2026.
- [8] AtomicJar, Inc. Testcontainers for Java, 2024. [Online] <https://java.testcontainers.org/>. Accessed June 2026.
- [9] Red Hat, Inc. Arquillian Container Testing Framework, 2024. [Online] <https://arquillian.org/>. Accessed June 2026.
- [10] REST Assured Contributors. REST Assured — Java DSL for testing REST services, 2024. [Online] <https://rest-assured.io/>. Accessed June 2026.
- [11] Postman, Inc. Postman API Platform, 2024. [Online] <https://www.postman.com/>. Accessed June 2026.

- [12] Postman, Inc. Newman — a Postman command-line collection runner, 2024. [Online] <https://github.com/postmanlabs/newman>. Accessed June 2026.
- [13] Twitter Engineering. Diffy — Automatically catch bugs in service-oriented architectures, 2015. [Online] https://blog.twitter.com/engineering/en_us/a/2015/diffy-testing-services-without-writing-tests. Accessed June 2026.
- [14] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [15] Jenkins Project. Jenkins — The leading open source automation server, 2024. [Online] <https://www.jenkins.io/>. Accessed June 2026.
- [16] GitHub Inc. GitHub Actions Documentation, 2024. [Online] <https://docs.github.com/actions>. Accessed June 2026.
- [17] GitLab Inc. GitLab CI/CD documentation, 2024. [Online] <https://docs.gitlab.com/ci/>. Accessed June 2026.
- [18] GitLab Inc. GitLab Runner documentation, 2024. [Online] <https://docs.gitlab.com/runner/>. Accessed June 2026.
- [19] GitLab Inc. GitLab Self-Managed — Install and configure, 2024. [Online] <https://about.gitlab.com/install/>. Accessed June 2026.
- [20] Roy T. Fielding. Architectural Styles and the Design of Network-based Software Architectures. In *Ph.D. dissertation, University of California, Irvine*, 2000. Chapter 5 introduces REST.
- [21] OpenAPI Initiative. OpenAPI Specification, Version 3.1.0, 2021. [Online] <https://spec.openapis.org/oas/v3.1.0>. Accessed June 2026.
- [22] SmartBear Software. Swagger UI, 2024. [Online] <https://swagger.io/tools/swagger-ui/>. Accessed June 2026.
- [23] Oracle Corporation. *Java Platform, Standard Edition 21 API Specification*. Oracle Corporation, 2023. [Online] <https://docs.oracle.com/en/java/javase/21/>.
- [24] Eclipse Foundation. Jakarta RESTful Web Services 4.0, 2024. [Online] <https://jakarta.ee/specifications/restful-ws/4.0/>. Accessed June 2026.

- [25] Eclipse Foundation. Jakarta Persistence 3.2, 2024. [Online] <https://jakarta.ee/specifications/persistence/3.2/>. Accessed June 2026.
- [26] Eclipse Foundation. Eclipse MicroProfile, 2024. [Online] <https://microprofile.io/>. Accessed June 2026.
- [27] VMware, Inc. Spring Boot Reference Documentation, 2024. [Online] <https://docs.spring.io/spring-boot/index.html>. Accessed June 2026.
- [28] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, 2024. [Online] <https://www.postgresql.org/docs/16/>. Accessed June 2026.
- [29] Oracle Corporation. MySQL 8.0 Reference Manual, 2024. [Online] <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed June 2026.
- [30] Red Hat, Inc. Hibernate ORM Documentation, 2024. [Online] <https://hibernate.org/orm/documentation/>. Accessed June 2026.
- [31] Brett Wooldridge. HikariCP — A high-performance JDBC connection pool, 2024. [Online] <https://github.com/brettwooldridge/HikariCP>. Accessed June 2026.
- [32] The Apache Software Foundation. Apache Tomcat documentation, 2024. [Online] <https://tomcat.apache.org/>. Accessed June 2026.
- [33] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014. [Online] <https://git-scm.com/book/en/v2>.
- [34] Docker, Inc. Docker Documentation, 2024. [Online] <https://docs.docker.com/>. Accessed June 2026.
- [35] Docker, Inc. Compose Specification, 2024. [Online] <https://compose-spec.io/>. Accessed June 2026.
- [36] Jason A. Donenfeld. WireGuard: Next Generation Kernel Network Tunnel, 2020. [Online] <https://www.wireguard.com/>. Accessed June 2026.